Microcontroladores

Profesor: Kalun José Lau Gan

Semestre 2026-2

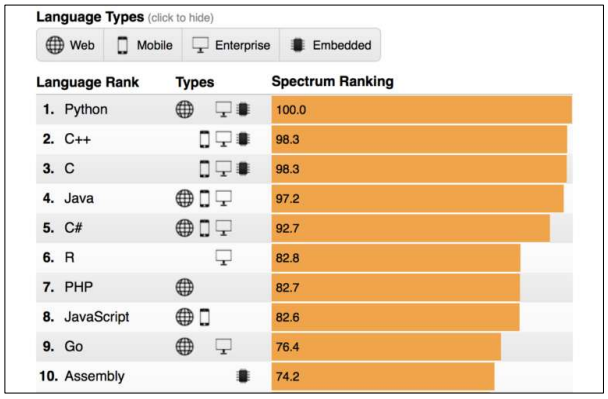
Semana 9-10

Coordinaciones previas:

- Con respecto al TP: Las diapositivas tienen que ser basadas en el informe, tener coherencia (si lo que se expone tiene sentido), contexto, etc.
- Lineamientos del TF: recordar que el TF es la continuación del TP (el desarrollo del proyecto)
  - Se añadirán los capítulos de Desarrollo, Validación, Resultados, Conclusiones y Recomendaciones.
- Deberán de hacer las correcciones de su TP tanto INF como PPT de acuerdo a la retroalimentación brindada y formará parte de la evaluación de PC2.
- Evaluaciones pendientes: LB3 (sem12), PC2 (sem14), DD (sem15), TF (sem16).
- Se tendrá espacios de tiempo en la sección de laboratorio para que se resuelvan interrogantes con respecto a TF

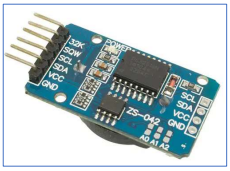
¿Preguntas previas?

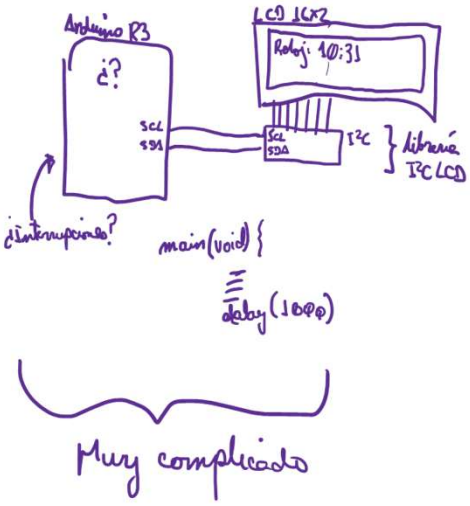
- ¿El hecho de usar C lo hace más fácil?
  - El proceso de diseño es el mismo tanto para lenguaje Assembler como en lenguaje C, se tiene que implementar primero el hardware, desarrollar el algoritmo y finalmente aterrizar el algoritmo en un lenguaje de programación.
  - El lenguaje XC va a permitir el uso de librerías y funciones especializadas, además de manejar mejor las operaciones matemáticas y manipulación de datos.
  - El lenguaje XC es mas familiar debido a que es un lenguaje orientado al desarrollador (alto nivel) y no a la máquina (bajo nivel)

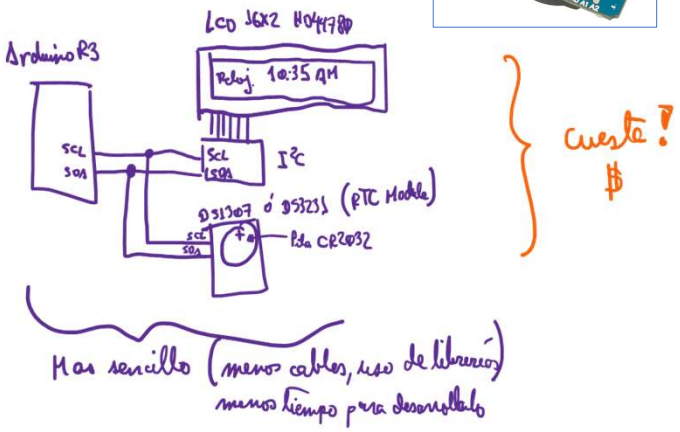


Quiero hacer un reloj, pero solo se desarrollar aplicaciones con Arduino

- El reloj debe de ser preciso



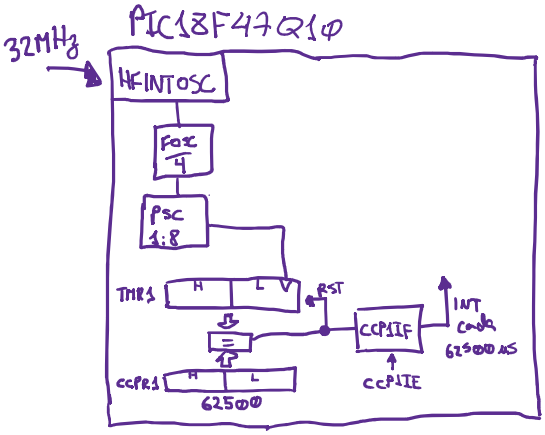


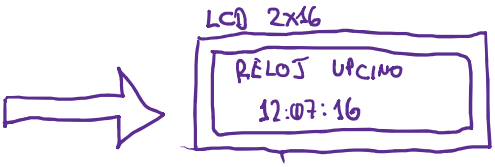


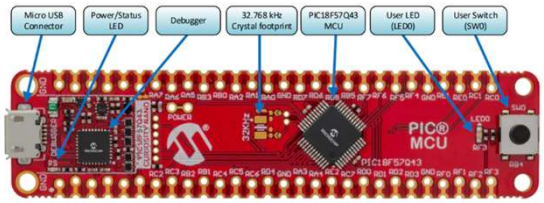
Ahora, vemos como hacer un reloj con microcontrolador PIC

- El reloj debe de ser preciso:
  - Usar XTAL 32.768KHz con el Timer1
  - Usar HFINTOSC a 32MHz y subsiguientes divisiones de dicha frecuencia









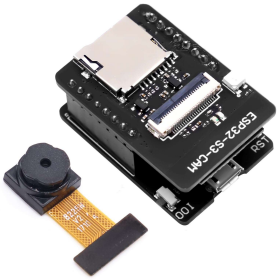
Preguntas previas:

- En el ejemplo de la semana pasada, el estar haciendo el cambio de velocidad me dí cuenta que demora en actualizar dicho cambio de velocidad aún utilizando interrupciones.
  - Esto se debe a que muy probablemente hayas empleado __delay_ms() para realizar el periodo de parpadeo de la aplicación. El CPU igual va a terminar de ejecutar dicha función aún haya habido una interrupción. Esto se nota mas cuando los retardo son muy prolongados.
  - Se recomienda para estos casos emplear un temporizador (ej Timer0) para hacer la labor de temporizado del tiempo y cada vez que ocurra una interrupción se actualiza el periodo de cuenta del temporizador y reiniciar su cuenta.

Preguntas previas:

- Como se haría para implementar un sistema electrónico basado en PDI para detectar una persona?
  - Utilizar una SBC como un Raspberry Pi para hacer la aplicación
  - La ventaja de usar RPi es que tiene un puerto dedicado para conectar una cámara y/o puertos USB.
  - La RPi funciona con un sistema operativo.
  - La solución es cara.
- Otra alternativa mas barata: ESP32 CAM



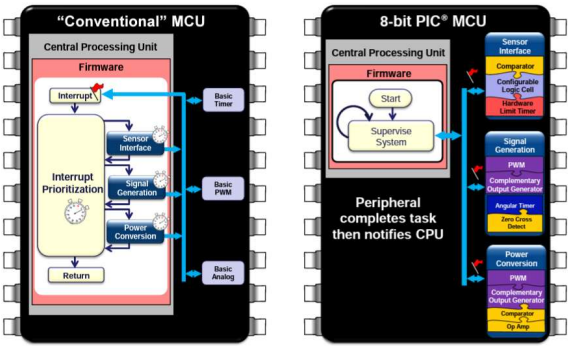


Agenda

- Tecnología de periféricos independientes de Microchip
- Lenguajes de alto nivel en el desarrollo con microcontroladores
  - El XC8 de Microchip
- Herramientas de apoyo al desarrollo
  - MCC (Microchip Code Configurator)
- Mi primer proyecto en lenguaje XC8 de alto nivel en el MPLAB X IDE
- Uso del LCD alfanumérico 16x2 HD44780
- El conversor ADC del PIC18F47Q10
- El conversor DAC del PIC18F47Q10

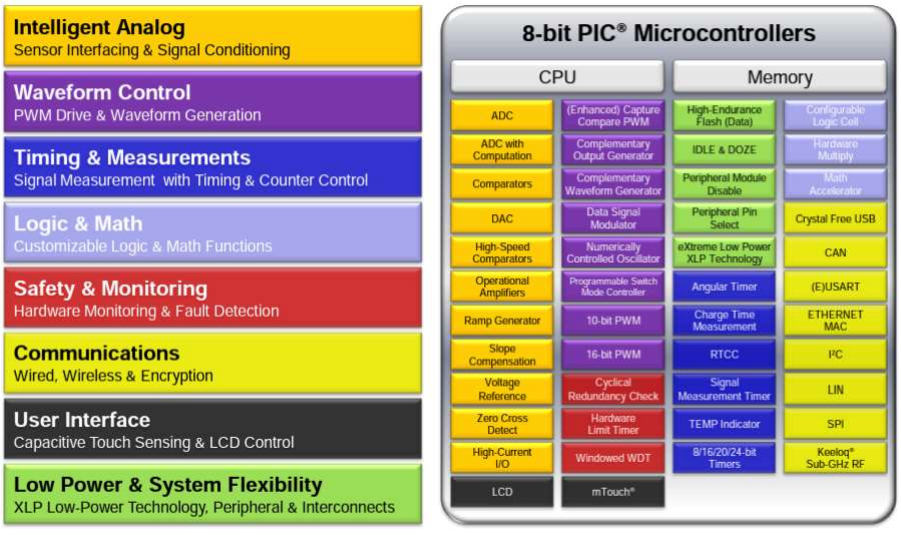
Tecnología de Periféricos Independientes (Core-Independent Peripherals – CIP)

- Independizando el funcionamiento de los periféricos en el microcontrolador se logra un mejor rendimiento del CPU
- Los periféricos independientes administran sus tareas sin código ni supervisión del CPU para mantener su funcionamiento

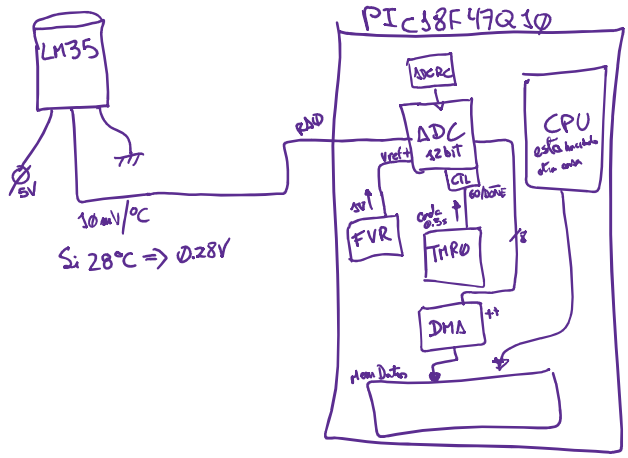


Tecnología de Periféricos Independientes (Core-Independent Peripherals – CIP)

- Dependiendo del modelo y familia el microcontrolador tendrá un conjunto de periféricos CIP
- Los CIPs poseen conexiones adicionales con otros CIP para lograr la independencia del CPU
- Estos CIP se apoyan en el sistema de interconexión de pines (PPS)

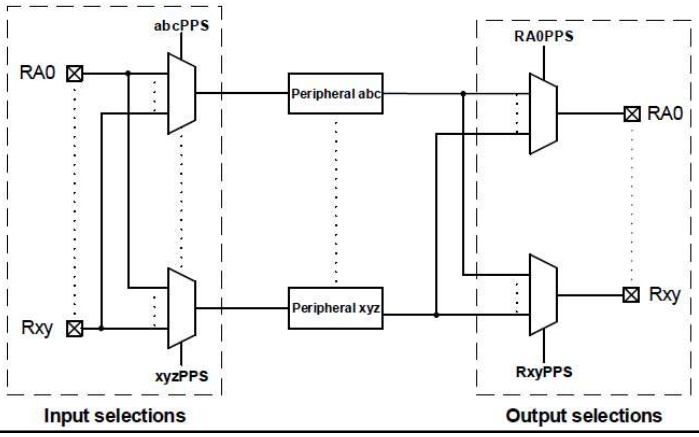


Ejemplo de CIP: Sensado de temperatura



El PPS en el PIC18F47Q10

- Referencia: cap 18 del datasheet
- Sistema de asignación personalizada de señales de E/S desde o hacia los periféricos.
- Tener en consideración las tablas “PPS Inputs” y “PPS Outputs” para las asignaciones por defecto y el alcance de la personalización



Panorama de los lenguajes de alto nivel para microcontroladores

Cada dispositivo microcontrolador tendrá sus propias plataformas de lenguaje de programación, no se pueden intercambiar de manera directa.

Va a depender del desarrollador del lenguaje de alto nivel para determinado microcontrolador (No todos los desarrolladores soportan todos los dispositivos de un fabricante)

Hay varias compañías que desarrollan lenguajes de alto nivel para un microcontrolador.

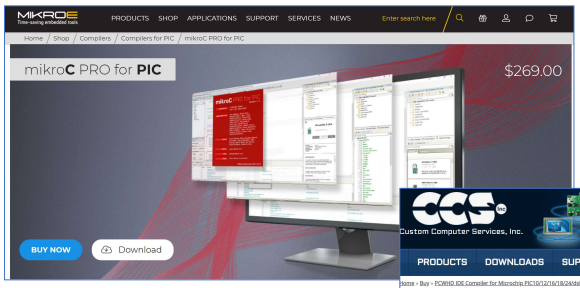
Problema de compatibilidad entre distintas plataformas de desarrollo aún empleando un mismo lenguaje.

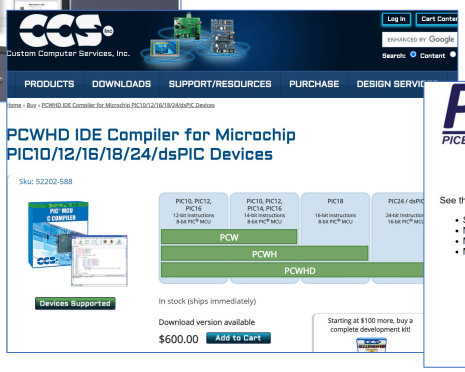
- Basic
- C
- Java (muy poco utilizado y soportado)
- Python (Rpi Pico RP2040 y el RP2350, y el ESP32)

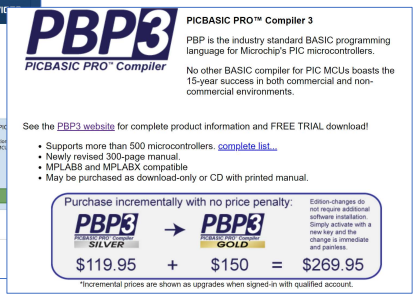


Panorama de los lenguajes de alto nivel para microcontroladores

- Los compiladores de lenguajes de alto nivel (de terceros) mas empleados en la actualidad.
- Estas empresas ofrecen versiones DEMO para que el usuario pueda probar las bondades de cada uno.
- Cada empresa ofrece un pack de librerías especializadas para crear aplicaciones con periféricos especializados en el menor tiempo.

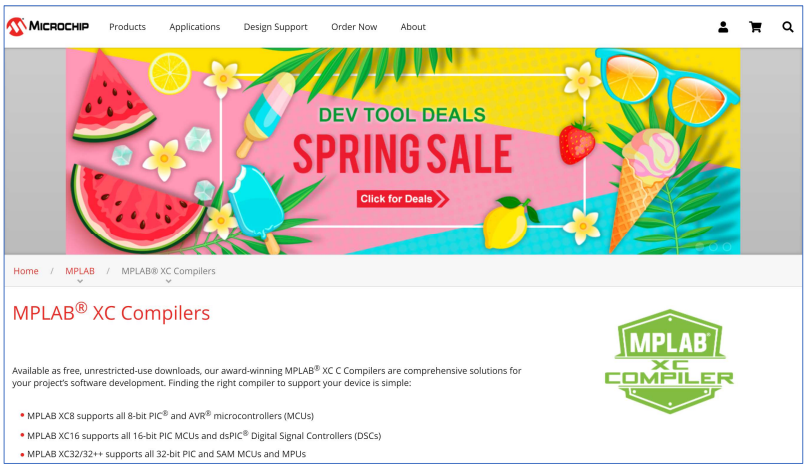






El compilador XC de Microchip

- Disponible versión gratis sin límite.
- Tres versiones:
  - XC8 – PIC10, PIC12, PIC16, PIC18
  - XC16 – PIC24, dsPIC
  - XC32 – PIC32



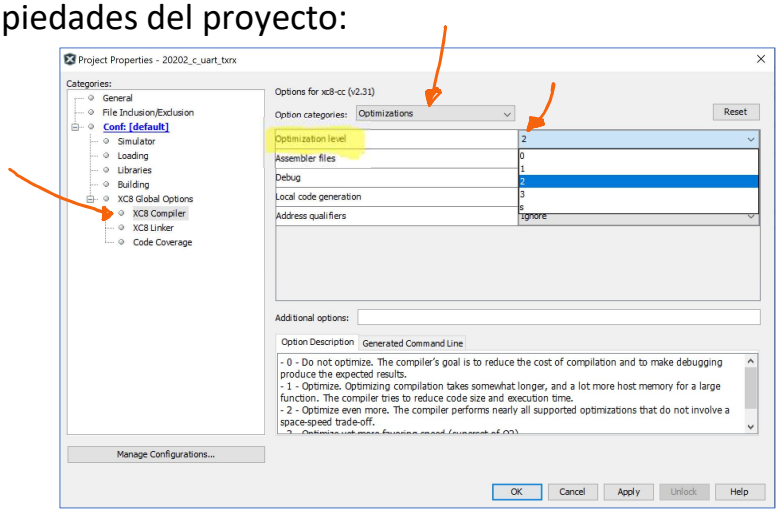
Optimización en el compilador XC

- **Optimization levels:**
  - Level 0: Fastest compilation time, minimal optimizations
  - Level 1: Optimizations with low debugging impact
  - Level 2: All optimizations that give the best balance between speed and size
  - Level 3: Program execution will be as fast as possible
  - Level s: Code size will be as small as possible
- **Where available use:**
  - Use Whole-program and Link-time setting
  - Procedural abstraction

- Por defecto el nivel de optimización esta en 0.

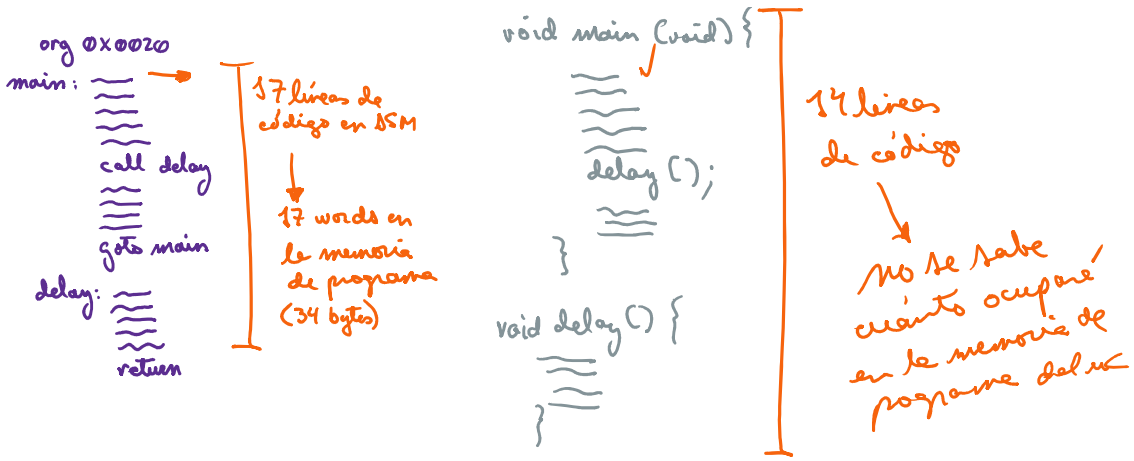
Optimización en el compilador XC

- Para acceder a las opciones de optimización ingresamos a las propiedades del proyecto:



Eficiencia de código en Assembler vs C

¿Por qué Assembler y por qué C?

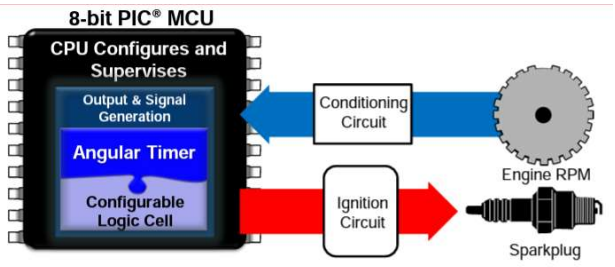


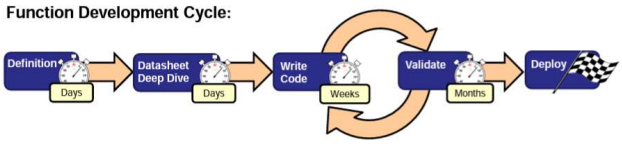
El Microchip Code Configurator (MCC)

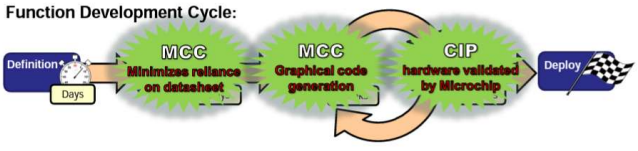
- Entorno de programación/configuración gráfico
- Interface intuitiva para desarrollos rápidos de prototipos
- Configuración automatizada para periféricos y funciones
  - Minimiza el uso de la hoja técnica para configuraciones
  - Reduce el esfuerzo y tiempo dedicado a la configuración
- Como desventaja: Va a ocupar mas espacio en la memoria de programa del micro debido a que incluye procedimientos que no necesariamente va a ejecutar el CPU.

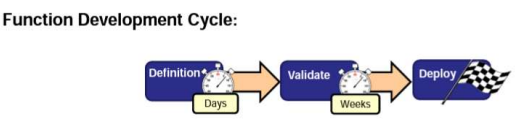
El Microchip Code Configurator (MCC)

- Caso de uso







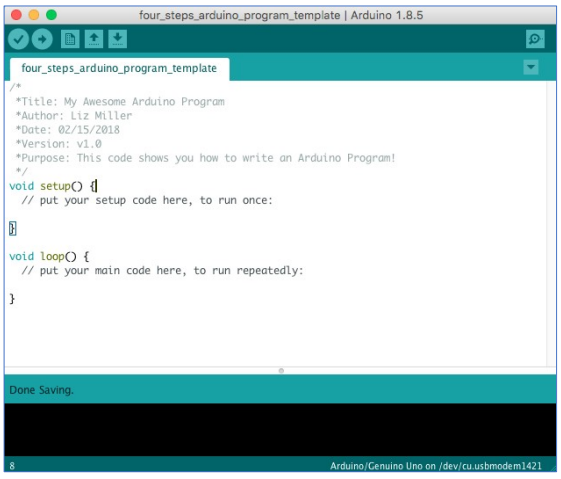


MPLAB X: Creación de un proyecto en XC8 (lenguaje de alto nivel)

- Link del manual de XC8:
  - http://ww1.microchip.com/downloads/en/devicedoc/50002053g.pdf
- Link de descarga del XC8 v4.00
  - https://www.microchip.com/mplabxc8windows

Referencia de plantilla:

- La plantilla para los códigos en C se ha basado en la plantilla de programas en Arduino IDE:
  - Uso de función setup() donde se coloca los aspectos iniciales de configuración antes de correr el programa de la aplicación. En XC8 crearemos una función similar. Por ejemplo configuración()
  - Uso de función loop() donde se detalla el programa de la aplicación. En XC8 usaremos la función main() donde dentro llamaremos a la función configuración antes de detallar el programa de usuario.



Plantilla de código en XC8:

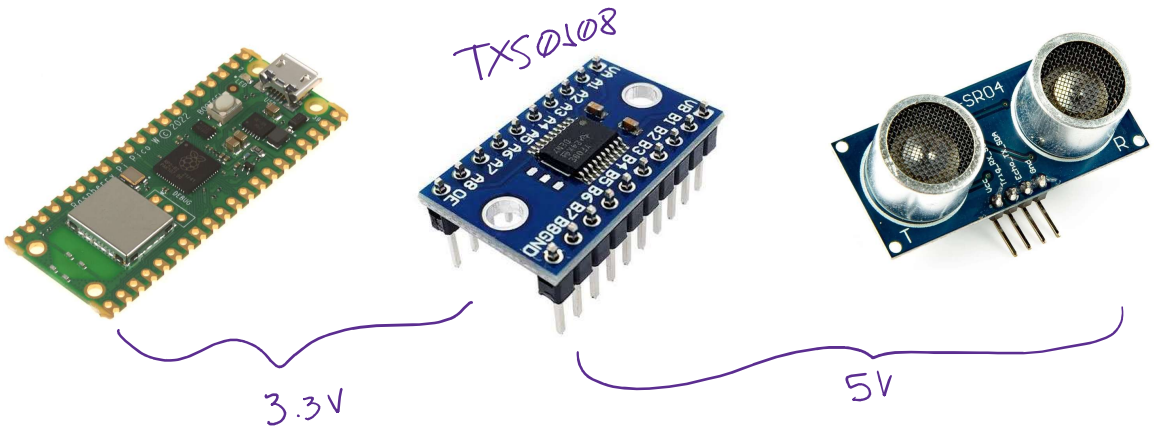
```c
#include <xc.h>

#define _XTAL_FREQ 48000000UL   //Frecuencia de trabajo 48MHz

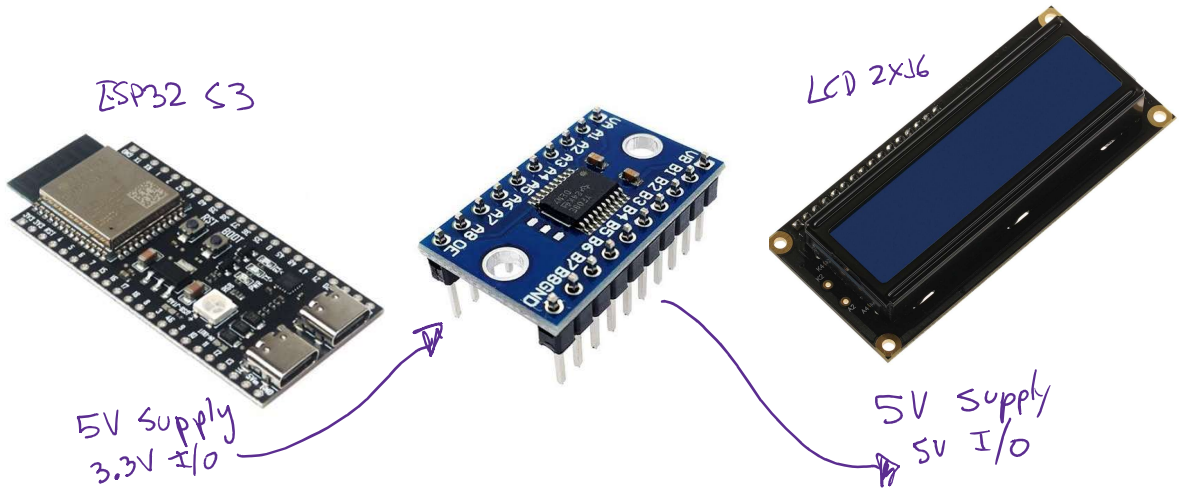
void configuracion(void) {
    //Aqui colocas las configuraciones iniciales
}

void main(void) {
    configuracion();
    while (1) {
        //Tu programa de usuario
    }
}
```

Conversores de niveles lógicos 5V a 3.3V

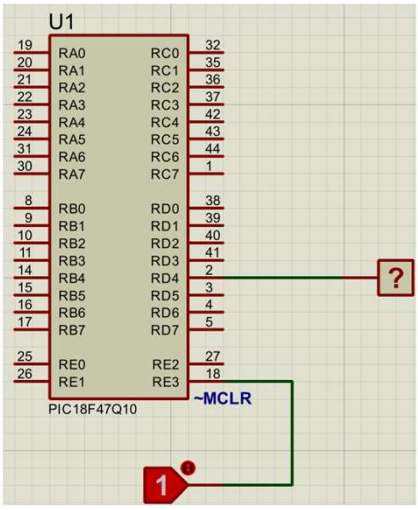


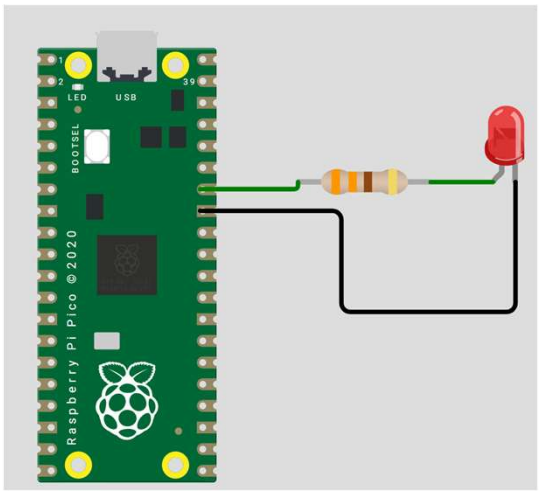
LCD 2x16 conectado a la Raspberry Pi Pico 2W



Ejemplo 2026-2 Sem09: Prender y apagar un LED

- Circuitos de prueba con PIC18F47Q10 y RP2350 (Raspberry Pi Pico)





Ejemplo 2026-2 Sem09: Prender y apagar un LED

- Programas de prueba en XC8 para PIC18F47Q10 y en Arduino para RP2350

```c
#include <xc.h>
#include "cabecera.h"
#define _XTAL_FREQ 4000000UL

void configuro(void){
    //conf el modulo de oscilador
    OSCCON1 = 0x60;      //NOSC=HFINTOSC NDIV=1:1
    OSCFRQ = 0x02;       //HFINTOSC a 4MHz
    OSCEN = 0x40;        //HFINTOSC enabled
    //conf el puerto de E/S
    TRISDbits.TRISD4 = 0;    //RD4 como salida
    ANSELDbits.ANSELD4 = 0; //Rd4 como digital
}

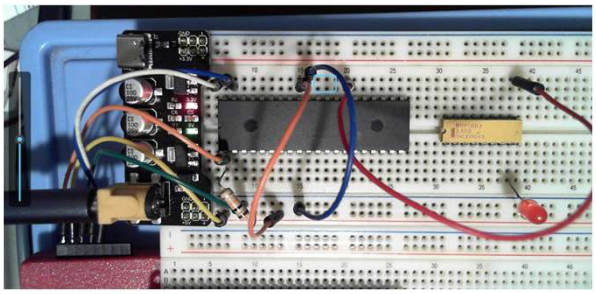
void main(void) {
    configuro();
    while(1){
        LATDbits.LATD4 = 1;
        __delay_ms(200);
        LATDbits.LATD4 = 0;
        __delay_ms(200);
    }
}
```

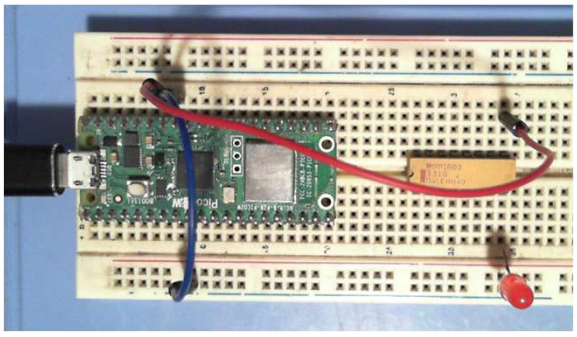
```c
void setup() {
  // put your setup code here, to run once:
  Serial1.begin(115200);
  Serial1.println("Hello, Raspberry Pi Pico!");
  pinMode(28, OUTPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  delay(1); // this speeds up the simulation
  while(1){
    digitalWrite(28, HIGH);
    delay(500);
    digitalWrite(28, LOW);
    delay(500);
  }
}
```

Ejemplo 2026-2 Sem09: Prender y apagar un LED

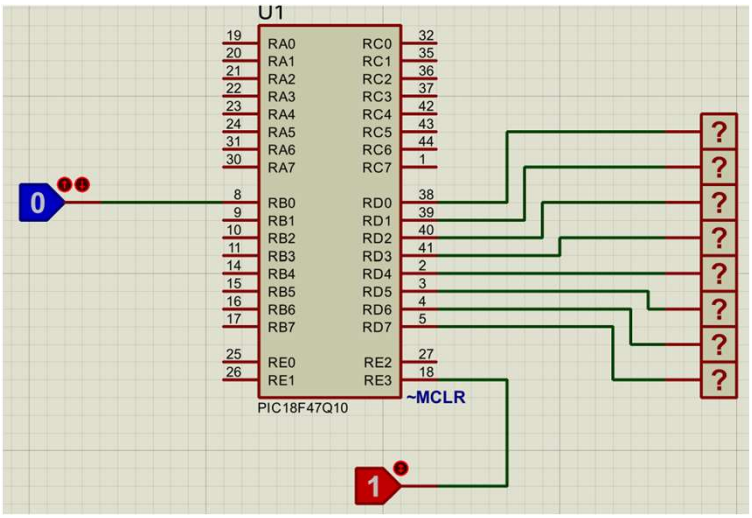
- Implementaciones en físico





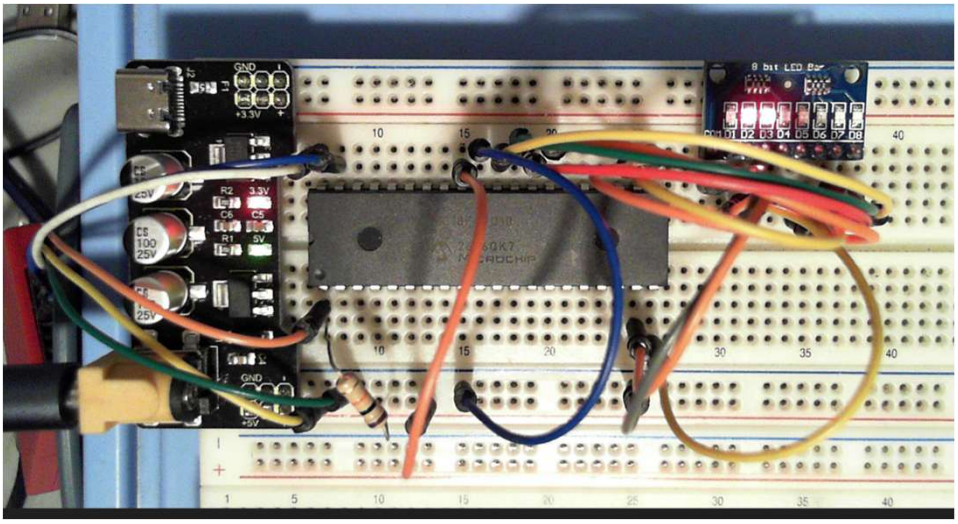
Ejemplo 2026-2: Auto fantástico

- Empleando el puerto D para los ocho LEDs



Ejemplo 2026-2: Auto fantástico

- Empleando el puerto D para los ocho LEDs



Ejemplo 2026-2: Auto fantástico

- Empleando el puerto D para los ocho LEDs

```c
#include <xc.h>
#include "cabecera.h"
#define _XTAL_FREQ 4000000UL
#define LEDS LATD

void configuro(void){
    //conf el modulo de oscilador
    OSCCON1 = 0x60;      //NOSC=HFINTOSC NDIV=1:1
    OSCFRQ = 0x02;       //HFINTOSC a 4MHz
    OSCEN = 0x40;        //HFINTOSC enabled
    //conf el puerto de E/S
    TRISD = 0x00;        //todo RD como salida
    ANSELD = 0x00;       //todo RD como digital
}

const unsigned char valores[] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};

void main(void) {
    configuro();
    while(1){
        for(unsigned char x_var=0;x_var<8;x_var++){
            LEDS = valores[x_var];
            __delay_ms(100);
        }
        for(unsigned char x_var=0;x_var<6;x_var++){
            LEDS = valores[6-x_var];
            __delay_ms(100);
        }
    }
}
```

Ejemplo 2026-2: Auto fantástico

```c
#include <xc.h>
#include "cabecera.h"
#define _XTAL_FREQ 4000000UL
#define LEDS LATD

void configuro(void){
    //conf el modulo de oscilador
    OSCCON1 = 0x60;      //NOSC=HFINTOSC NDIV=1:1
    OSCFRQ = 0x02;       //HFINTOSC a 4MHz
    OSCEN = 0x40;        //HFINTOSC enabled
    //conf el puerto de E/S
    TRISD = 0x00;        //todo RD como salida
    ANSELD = 0x00;       //todo RD como digital
    ANSELBbits.ANSELB0 = 0; //RB0 como digital
    WPUBbits.WPUB0 = 1;     //RB0 pullup enabled
}

const unsigned char valores[] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};
//RB0 como polling para cambiar la velocidad del desplazamiento
void main(void) {
    configuro();
    while(1){
        for(unsigned char x_var=0;x_var<8;x_var++){
            LEDS = valores[x_var];
            if(PORTBbits.RB0 == 0){
                __delay_ms(50);
            }
            else{
                __delay_ms(300);
            }
        }
        for(unsigned char x_var=0;x_var<6;x_var++){
            LEDS = valores[6-x_var];
            if(PORTBbits.RB0 == 0){
                __delay_ms(50);
            }
            else{
                __delay_ms(300);
            }
        }
    }
}
```

- Añadiendo funcionalidad de cambio de velocidad de desplazamiento empleando la entrada RB0 (modo polling)

Ejemplo 2026-2: Auto fantástico

- Ahora usando la INT0 para hacer el cambio de velocidad del desplazamiento

```c
#include <xc.h>
#include "cabecera.h"
#define _XTAL_FREQ 4000000UL
#define LEDS LATD

void configuro(void){
    //conf el modulo de oscilador
    OSCCON1 = 0x60;      //NOSC=HFINTOSC NDIV=1:1
    OSCFRQ = 0x02;       //HFINTOSC a 4MHz
    OSCEN = 0x40;        //HFINTOSC enabled
    //conf el puerto de E/S
    TRISD = 0x00;        //todo RD como salida
    ANSELD = 0x00;       //todo RD como digital
    ANSELBbits.ANSELB0 = 0; //RB0 como digital
    WPUBbits.WPUB0 = 1;     //RB0 pullup enabled
    //conf de las interrupciones
    INTCONbits.INT0EDG = 0; //deteccion de la INT0 sea flanco descendente
    PIE0bits.INT0IE = 1;    //INT0 habilitada
    INTCONbits.GIE = 1;     //GIE habilitado
}

const unsigned char valores[] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};
__bit estado = 0;
```

```c
void main(void) {
    configuro();
    while(1){
        for(unsigned char x_var=0;x_var<8;x_var++){
            LEDS = valores[x_var];
            if(estado == 0){
                __delay_ms(50);
            }
            else{
                __delay_ms(300);
            }
        }
        for(unsigned char x_var=0;x_var<6;x_var++){
            LEDS = valores[6-x_var];
            if(estado == 0){
                __delay_ms(50);
            }
            else{
                __delay_ms(300);
            }
        }
    }
}

void __interrupt() INT0_ISR(void){
    PIR0bits.INT0IF = 0;        //bajaamos la bandera INT0IF
    estado = ~estado;
}
```

Ejemplo 2026-2: Auto fantástico

- Usando el RP2350

sketch.ino | diagram.json | Library Manager

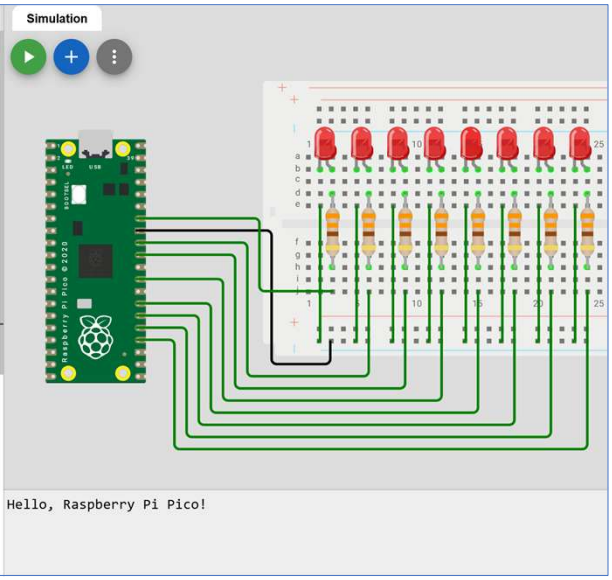
```c
const unsigned char LED_PINS[8] = {28, 27, 26, 22, 21, 20, 19, 18};
const unsigned char auto_fant[8] = {0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80};

void setup() {
  // put your setup code here, to run once:
  Serial1.begin(115200);
  Serial1.println("Hello, Raspberry Pi Pico!");
  //poner todos los puertos donde estan conectados los LED como salida
  for(unsigned char x_var=0;x_var<8;x_var++){
    pinMode(LED_PINS[x_var], OUTPUT);
  }
}

void LED_UPD(unsigned char dato){
  for(unsigned char y_var=0;y_var<8;y_var++){
    bool estado = (dato >> y_var) & 1;
    digitalWrite(LED_PINS[y_var], estado ? HIGH : LOW);
  }
}

void loop() {
  while(1){
    for(unsigned char z_var=0;z_var<8;z_var++){
      LED_UPD(auto_fant[z_var]);
      delay(300);
    }
    for(unsigned char z_var=0;z_var<6;z_var++){
      LED_UPD(auto_fant[6-z_var]);
      delay(300);
    }
  }
}
```

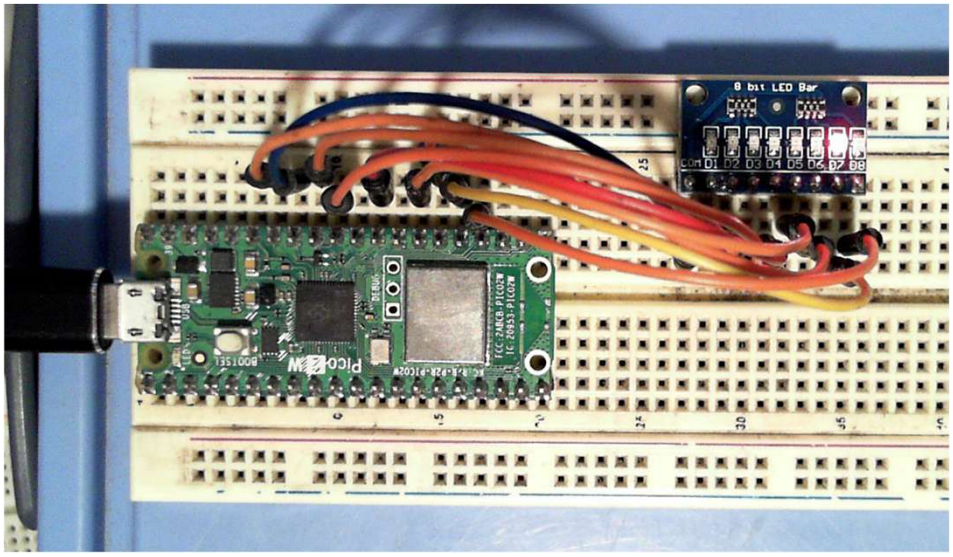
Simulation



Hello, Raspberry Pi Pico!

Ejemplo 2026-2: Auto fantástico

- Usando el RP2350

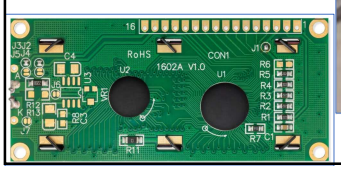


El LCD alfanumérico HD44780

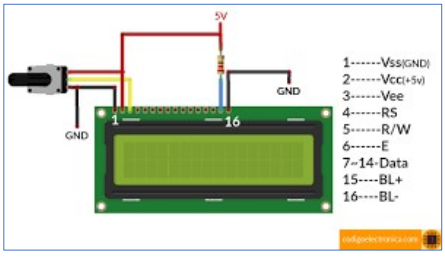
- Basado en el controlador Hitachi HD44780A
- Diferentes tamaños, desde 1x8 hasta 4x40
- Interface paralela de datos (4 ó 8 bits)
- Tiene control de contraste y luz de fondo
- Posee un ROM de caracteres predefinidos



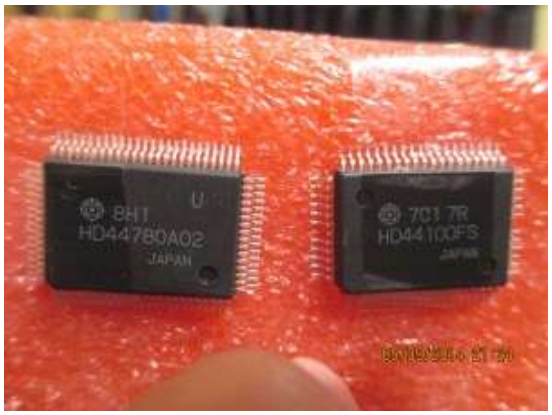








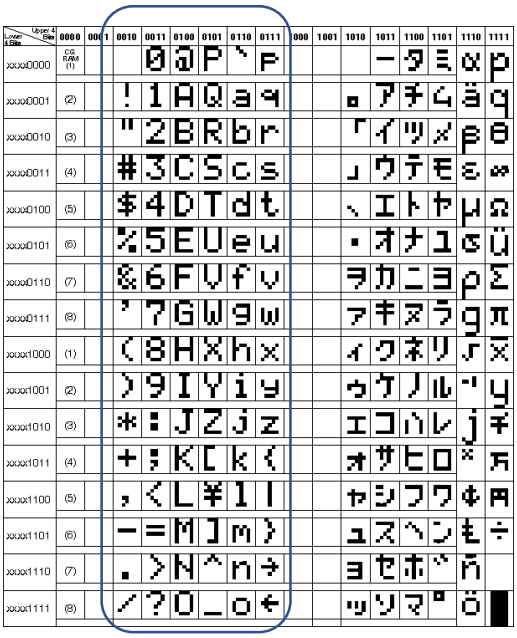
The holy grail of LCD controllers: HD44780A02





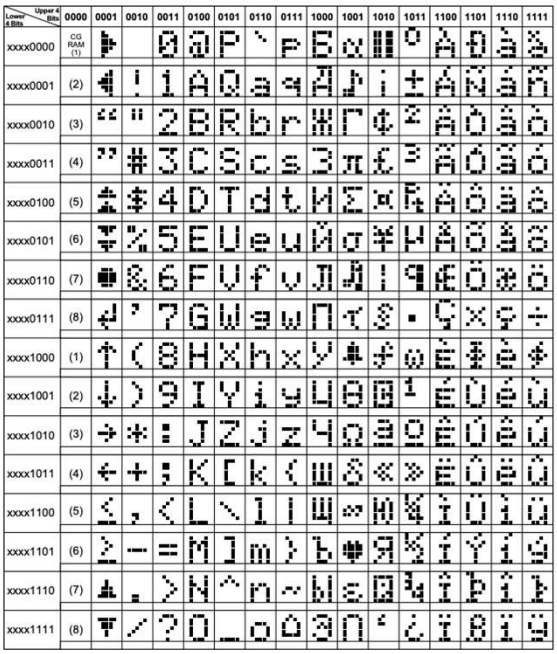
El LCD alfanumérico HD44780

- ROM de caracteres A00:
  - Muy similar al código ASCII en 7 bits
  - El símbolo de grado (°) en ASCII es Alt+0167, en el ROM de caracteres del HD44780 es 0xDF
  - El símbolo “ñ” en ASCII es Alt+164, en el ROM de caracteres del HD44780 es 0xEE
  - Capacidad de ocho caracteres personalizados (CGRAM 0x00-0x07)
  - Para enviar un carácter directamente al display se emplea la función ENVIA_CHAR() (librería S_SAL)



El LCD alfanumérico HD44780

- ROM de caracteres A02:
  - Posee caracteres adicionales que son utilizados en occidente.
  - Los HD44780 con ROM code A02 son extremadamente escasos.



El LCD alfanumérico HD44780

- Tabla de caracteres ASCII de 7 bits
- Ref. http://www.asciitable.com

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

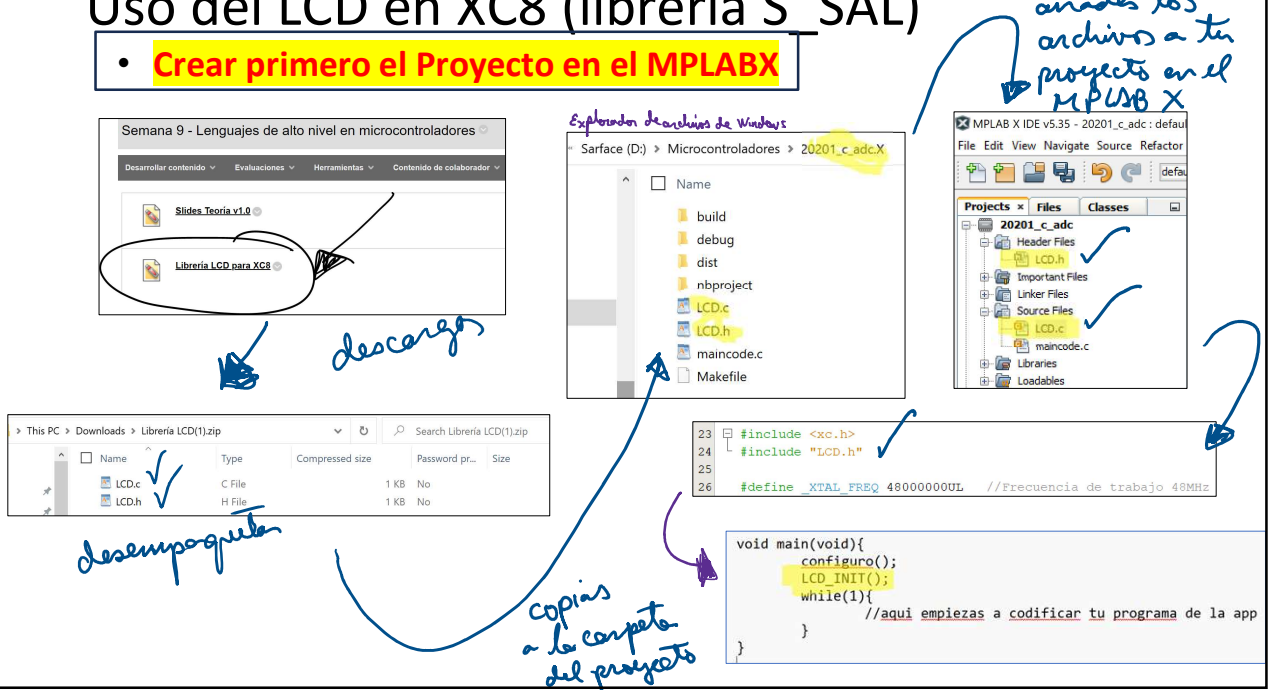
Source: www.LookupTables.com

El LCD alfanumérico HD44780

- Referencia: Hoja técnica del HD44780
  - http://academy.cba.mit.edu/classes/output_devices/44780.pdf
- Para trabajar con el display se ha creado una librería de comandos (desarrollado por Sergio Salas y Kalun Lau) en la cual posee las siguientes características:
  - Interface de 4 bits
  - Comandos para: Limpiar pantalla, ocultar cursor, pasar de línea, caracteres personalizados, etc.
  - Puerto D empleado (RD0→RS, RD1→RW, RD2→E, RD4→D4, RD5→D5, RD6→D6, RD7→D7)
  - Tener en cuenta FOSC especificado dentro de la librería (4MHz). Esto se encuentra en LCD.h en el #define _XTAL_FREQ
- Video de manipulación de LCD sin microcontrolador:
  - https://www.youtube.com/watch?v=cXpeTxC3_A4

Uso del LCD en XC8 (librería S_SAL)

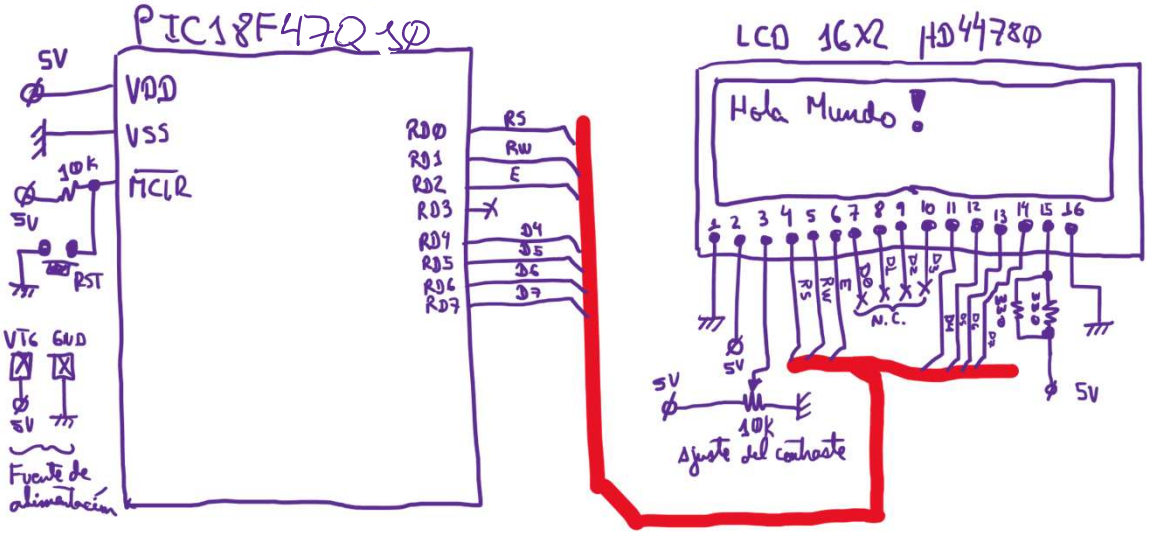
- **Crear primero el Proyecto en el MPLABX**



Uso de la librería LCD

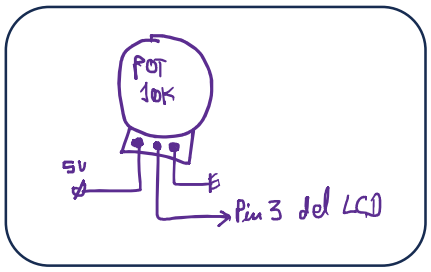
- ¿Tengo que hacer ese procedimiento de llamado de la librería por cada proyecto que haga?
  - Si, para cada proyecto que uses el LCD y la librería, tendrás que hacer ese procedimiento.
- ¿Esta librería la puedo usar para otros modelos de microcontrolador PIC18?
  - Si, para cualquier modelo de PIC18 siempre y cuando el lenguaje de programación sea XC8 de alto nivel y que tenga disponible el puerto D.

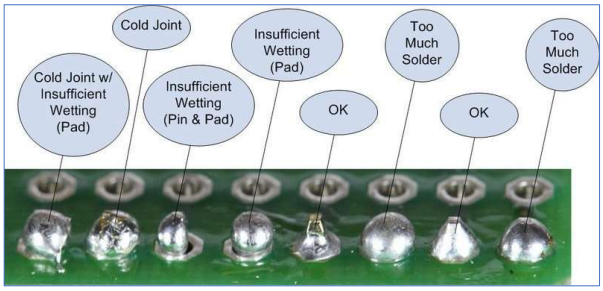
Circuito ejemplo de conexión del PIC18F47Q10 con el LCD



Recomendaciones en la interface del LCD con el microcontrolador

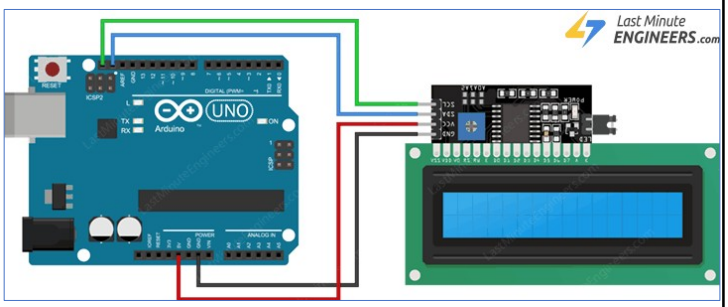
- Los pines deben de estar soldados correctamente!
- Seguir estrictamente el procedimiento para el llamado de las librerías.
- El potenciómetro debe de estar correctamente instalado





Interface del LCD 2x16 HD44780 en I2C

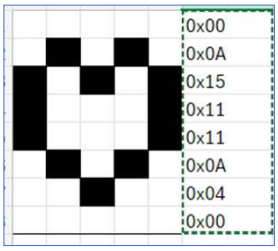
- Se le agrega un módulo adicional al LCD y permite la interface a dicho LCD usando I2C (comunicación serial), dicho módulo está basado en el circuito integrado PCF8574 (expansor de puertos vía I2C)
- Permite minimizar la cantidad de pines entre el LCD y el microcontrolador.
- Se requiere implementar una librería para poder lograr la interface
- Es más fácil, pero tiene un costo mayor



Caracteres personalizados en el LCD 16x2 HD44780

- Podemos incluir caracteres personalizados en nuestras visualizaciones.
- Cada caracter corresponde a una matriz de 8x5 (incluyendo el área del cursor)
- Disponible hasta **ocho** caracteres personalizados.
- Generador online de caracteres personalizados:
  - https://maxpromer.github.io/LCD-Character-Creator/

Procedimiento para caracteres personalizados



1. Diseñar el caracter personalizado y extraer los ocho datos

2. Declarar la constante global con los ocho datos del caracter personalizado

```
11    const unsigned char corazoncito[]={0x00,0x0A,0x15,0x11,0x11,0x0A,0x04,0x00};
```

```
void LCD_init(void){
    TRISD = 0x00;
    ANSELD = 0x00;
    __delay_ms(18);
    LCD_CONFIG();
    __delay_ms(19);
    BORRAR_LCD();
    CURSOR_HOME();
    CURSOR_ONOFF(OFF);
    GENERACARACTER(corazoncito, 0);
}
```

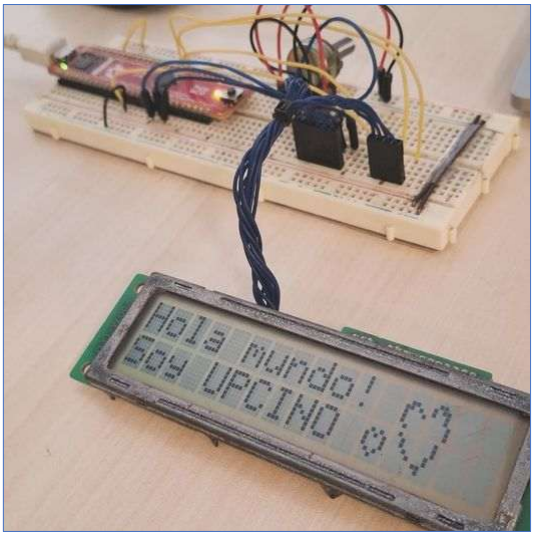
3. En la rutina de inicialización del LCD, llamar a la función GENERACARACTER para cargar el caracter personalizado a la memoria CGRAM del LCD

4. Para llamar al caracter personalizado, utilizar la función ENVIA_CHAR()

```
void main(void) {
    configuro();
    LCD_init();
    POS_CURSOR(1,0);
    ESCRIBE_MENSAJE("Hola mundo!", 11);
    POS_CURSOR(2,0);
    ESCRIBE_MENSAJE("Soy UPCINO!", 11);
    POS_CURSOR(2, 13);
    ENVIA_CHAR(0);
```

Caracteres personalizados

Visualización de una figura empleando multiples caracteres personalizados



H	I	J	K	L	M	N	O	P	Q	R	S
										0x00	0x00
										0x00	0x0C
										0x0C	0x16
										0x1A	0x12
										0x11	0x02
										0x10	0x01
										0x10	0x01
										0x08	0x01
										0x08	0x02
										0x04	0x02
										0x04	0x02
										0x02	0x04
										0x01	0x04
										0x00	0x18
										0x00	0x08
										0x00	0x00

Conversor analógica a digital (A/D ó ADC)

Revisar Capítulo 33 del datasheet

- Resolución:
- Cantidad de canales analógicos:
- Tiempo de adquisición:
- Rango de voltaje de entrada:
- ¿Cuáles son los valores límites de Vref+ y Vref-?
- Proceso de adquisición de una señal analógica
- ¿Interviene el teorema de muestreo?
- ¿Hay interrupciones?

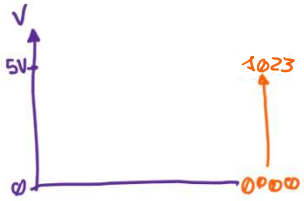
Conversor A/D (PIC18F47Q10)

- Resolución: 10bits (ADRESH:ADRESL)
  - Posee un bit ADFM (justificación del resultado)
  - Total 1024 escalones (rango de 0 a 1023)
- Cantidad de canales analógicos: 35
- Se lee un canal analógico a la vez, de manera secuencial
- ¿Interviene el teorema de muestreo? Si.
- Rango máximo de voltaje de entrada por canal: 0-5V?
  (Vref+ = VDD, Vref- = VSS)
- Las señales analógicas no deben bajar de 0V
- Proceso de adquisición de una señal analógica (ver datasheet)
- ¿Hay interrupciones? Si. (ADIE, ADIF)
- Posee módulo computacional →

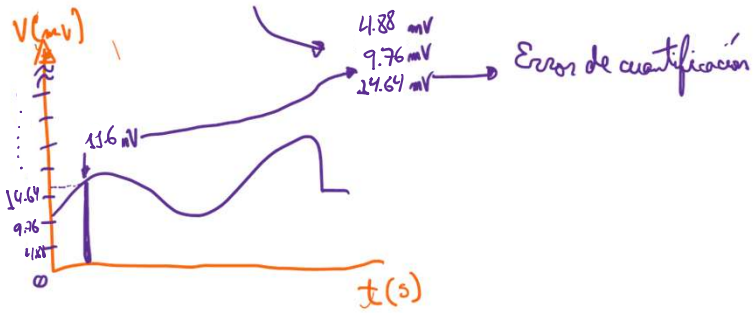
- Computation Features:
  - Averaging and low-pass filter functions
  - Reference comparison
  - 2-level threshold comparison
  - Selectable interrupts

Sensibilidad del A/D del PIC18F47Q10

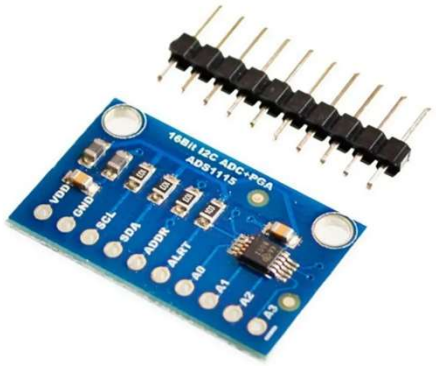
- Rango de la señal de entrada:
  - Asumiendo que Vref+ = 5V, Vref- = 0V
  - El rango es 0-5V
- Resolución es de 10bits (1024 escalones) ← 2^{10}
- Altura del escalón: $\frac{5}{1024} = 4.88\ mV$ //



Ejemplo:



Alternativa de A/D de mayor resolución



ADS1115 Módulo ADC I2C de 16 bits, 4 canales, PGA integrado

Precio incluye IGV

S/ 15.00

En stock

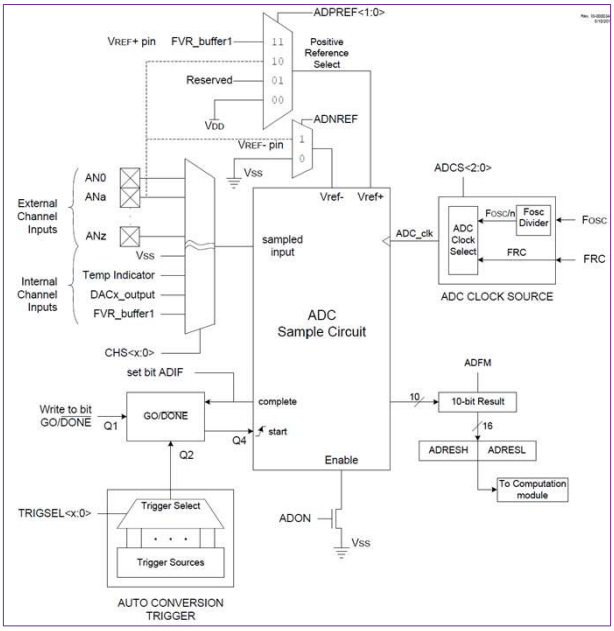
1 Añadir al carrito

SKU: EM14

Categoría: Sensores y módulos

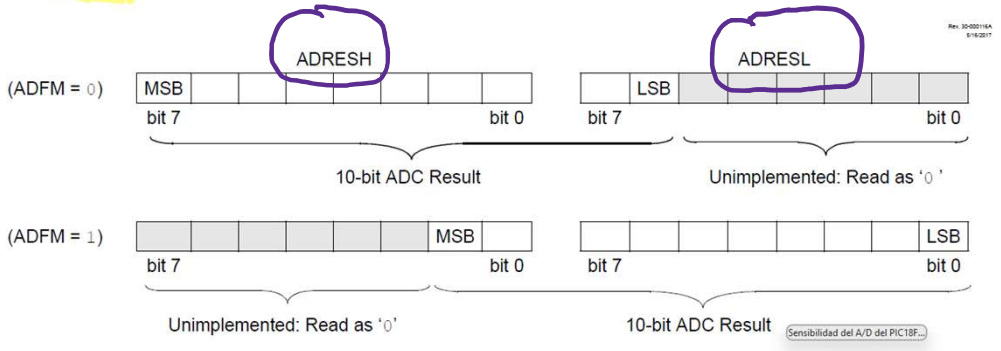
Etiquetas: adc, arduino, esp32, i2c

Conversor A/D: Diagrama de bloques



Conversor A/D: Justificación del resultado

Figure 33-3. 10-Bit ADC Conversion Result Format




```
unsigned int lectura = 0;      //lectura es una variable de 16 bits
lectura = (ADRESH << 8) + ADRESL;
```

Conversor A/D: Registros

33.6 Register Summary - ADC Control

ADCON1

Address	Name	Bit Pos.	7	6	5	4	3	2	1	0
0x00 ... 0x0F50	Reserved									
0x0F51	ADACT	7:0				ADACT[4:0]				
0x0F52	ADCLK	7:0			ADCS[5:0]					
0x0F53	ADREF	7:0				ADNREF			ADPREF[1:0]	
0x0F54	ADCON1	7:0	ADPPOL	ADIPEN	ADGPOL					ADDSEN
0x0F55	ADCON2	7:0	ADPSIS		ADCRS[2:0]		ADACLR		ADMD[2:0]	
0x0F56	ADCON3	7:0			ADCALC[2:0]		ADSOI		ADTMD[2:0]	
0x0F57	ADACQ	7:0	ADACQ[7:0]							
0x0F58	ADCAP	7:0				ADCAP[4:0]				
0x0F59	ADPRE	7:0	ADPRE[7:0]							
0x0F5A	ADPCH	7:0				ADPCH[5:0]				
0x0F5B	ADCON0	7:0	ADON	ADCONT		ADCS		ADFM		ADGO
0x0F5C	ADPREV	7:0	ADPREVL[7:0]							
		15:8	ADPREVH[7:0]							
0x0F5E	ADRES	7:0	ADRESL[7:0]							
		15:8	ADRESH[7:0]							
0x0F60	ADSTAT	7:0	ADAOV	ADUTHR	ADLTHR	ADMATH		ADSTAT[2:0]		
0x0F61	ADRPT	7:0	ADRPT[7:0]							
0x0F62	ADCNT	7:0	ADCNT[7:0]							
0x0F63	ADSTPT	7:0	ADSTPTL[7:0]							
		15:8	ADSTPTH[7:0]							
0x0F65	ADLTH	7:0	ADLTHL[7:0]							
		15:8	ADLTHH[7:0]							
0x0F67	ADUTH	7:0	ADUTHL[7:0]							
		15:8	ADUTHH[7:0]							
0x0F69	ADERR	7:0	ADERRL[7:0]							
		15:8	ADERRH[7:0]							
0x0F6B	ADACC	7:0	ADACCL[7:0]							
		15:8	ADACCH[7:0]							
0x0F6D	ADFLTR	7:0	ADFLTRL[7:0]							
		15:8	ADFLTRH[7:0]							

Conversor A/D: Registros

Name: ADCON0
Address: 0xF5B

ADC Control Register 0

Bit	7	6	5	4	3	2	1	0
	ADON	ADCONT		ADCS		ADFM		ADGO
Access	R/W	R/W		R/W		R/W		R/W/HC
Reset	0	0		0		0		0

Bit 7 – ADON ADC Enable bit

Value	Description
1	ADC is enabled
0	ADC is disabled

Bit 6 – ADCONT ADC Continuous Operation Enable bit

Value	Description
1	ADGO is retriggered upon completion of each conversion trigger until ADTIF is set (if ADSOI is set) or until ADGO is cleared (regardless of the value of ADSOI)
0	ADC is cleared upon completion of each conversion trigger

Bit 4 – ADCS ADC Clock Selection bit

Value	Description
1	Clock supplied from FRC dedicated oscillator
0	Clock supplied by Fosc, divided according to ADCLK register

Bit 2 – ADFM ADC results Format/alignment Selection

Value	Description
1	ADRES and ADPREV data are right justified
0	ADRES and ADPREV data are left justified, zero-filled

Bit 0 – ADGO ADC Conversion Status bit

Value	Description
1	ADC conversion cycle in progress. Setting this bit starts an ADC conversion cycle. The bit is cleared by hardware as determined by the ADCONT bit
0	ADC conversion completed/not in progress

Conversor A/D: Registros

Name: ADRES
Address: 0xF5E

ADC Result Register

Bit	15	14	13	12	11	10	9	8
	ADRESH[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
	ADRESL[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 15:8 – ADRESH[7:0] ADC Result Register bits. High bits

Value	Condition	Description
0x00,0x01,0x02,0x03	ADFM=1	Upper 2 bits of result
0 to 0xFF	ADFM=0	Upper 8 bits of result

Bits 7:0 – ADRESL[7:0] ADC Result Register bits. Lower bits

Value	Condition	Description
0 to 0xFF	ADFM=1	Lower 8 bits of result
0x00,0x40,0x80,0xC0	ADFM=0	Lower 2 bits of result

Conversor A/D: Registros

Name: ADPCH
Address: 0xF5A

ADC Positive Channel Selection Register

Bit	7	6	5	4	3	2	1	0
			ADPCH[5:0]					
Access			R/W	R/W	R/W	R/W	R/W	R/W
Reset			0	0	0	0	0	0

Bits 5:0 – ADPCH[5:0] ADC Positive Input Channel Selection bits
See Channel Selection Table for input selection details.

ADPCH	ADC Positive Channel Input
111111	Fixed Voltage Reference (FVR)[2]
111110	DAC1 output[1]
111101	Temperature Indicator[3]
111100	AVSS (Analog Ground)
100011–111011	Reserved. No channel connected.
100010	RE2/ANE2[1]
100001	RE1/ANE1[1]
100000	RE0/ANE0[1]
011111	RD7/AND7[1]
011110	RD6/AND6[1]
011101	RD5/AND5[1]
011100	RD4/AND4[1]
011011	RD3/AND3[1]
011010	RD2/AND2[1]
011001	RD1/AND1[1]
011000	RD0/AND0[1]
010111	RC7/ANC7
010110	RC6/ANC6
010101	RC5/ ANC5
010100	RC4/ ANC4
010011	RC3/ANC3
010010	RC2/ANC2
010001	RC1/ ANC1
010000	RC0/ANC0
001111	RB7/ANB7
001110	RB6/ANB6
001101	RB5/ANB5
001100	RB4/ ANB4
001011	RB3/ANB3
001010	RB2/ ANB2
001001	RB1/ ANB1
001000	RB0/ANB0
000111	RA7/ANA7
000110	RA6/ANA6
000101	RA5/ANA5
000100	RA4/ ANA4
000011	RA3/ ANA3
000010	RA2/ ANA2
000001	RA1/ ANA1
000000	RA0/ANA0

Note:
1. 40/44-pin devices only.

Conversor A/D: Registros

Name: ADREF
Address: 0xF53

ADC Reference Selection Register

Bit	7	6	5	4	3	2	1	0
				ADNREF			ADPREF[1:0]	
Access				R/W			R/W	R/W
Reset				0			0	0

Bit 4 – ADNREF ADC Negative Voltage Reference Selection bit

Value	Description
1	V_{REF}- is connected to external V_{REF}-
0	V_{REF}- is connected to AV_{SS}

Bits 1:0 – ADPREF[1:0] ADC Positive Voltage Reference Selection bits

Value	Description
11	V_{REF}+ is connected to internal Fixed Voltage Reference (FVR) module
10	V_{REF}+ is connected to external V_{REF}+
01	*Reserved*
00	V_{REF}+ is connected to V_{DD}

Conversor A/D: Procedimiento para configurar y adquirir un dato de un canal analógico

Configuración: (Recordar que el rango de voltaje de entrada es de 0V a 5V)

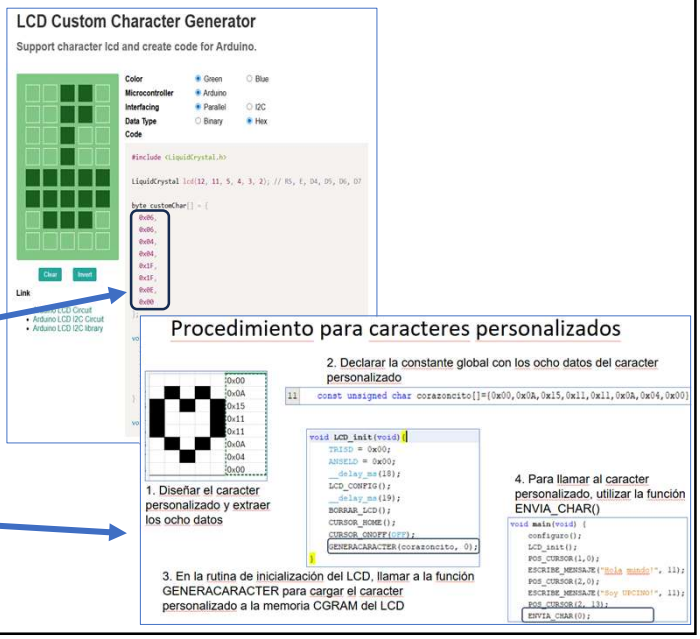
1. Establecer qué puerto será el que reciba la señal analógica
2. Configurar dicho puerto para que sea del tipo entrada y analógico (TRISx.y y ANSELx.y donde “x” es el puerto y “y” el pin)
3. Definir el bit FM: 0 para justificación del resultado a la izquierda, 1 para justificación a la derecha
4. Definir la base de tiempo del ADC con el bit CS
5. Habilitar el funcionamiento del ADC con el bit ADON

Adquisición (toma una muestra):

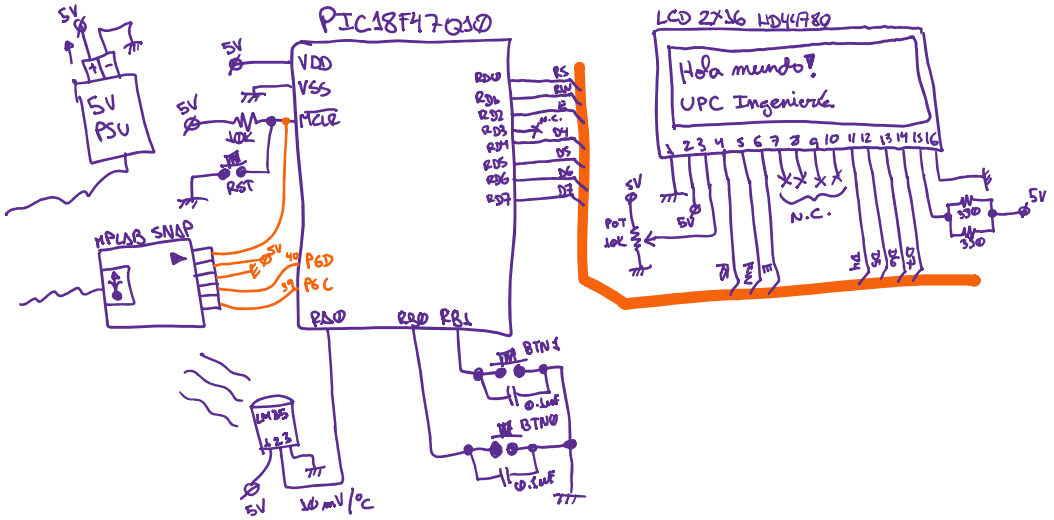
1. Seleccionar el canal de lectura con el registro ADPCH
2. Iniciar la conversión con el bit GO=1
3. Esperar a que el bit GO baje a cero (término de la conversión)
4. El resultado estará en el par de registros ADRESH:ADRESL

Caracteres personalizados

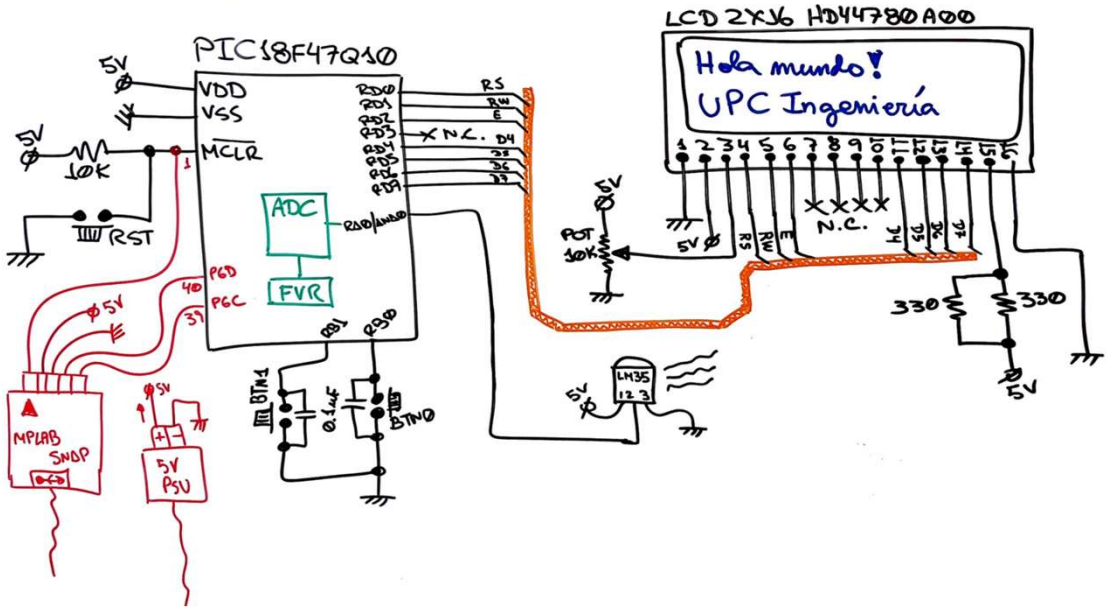
- Hasta 8 caracteres personalizados (0 al 7)
- Link: https://maxpromer.github.io/LCD-Character-Creator/
- Nos interesa solo los códigos generados en hexadecimal
- Seguir el procedimiento que se encuentra en la diapositiva 57



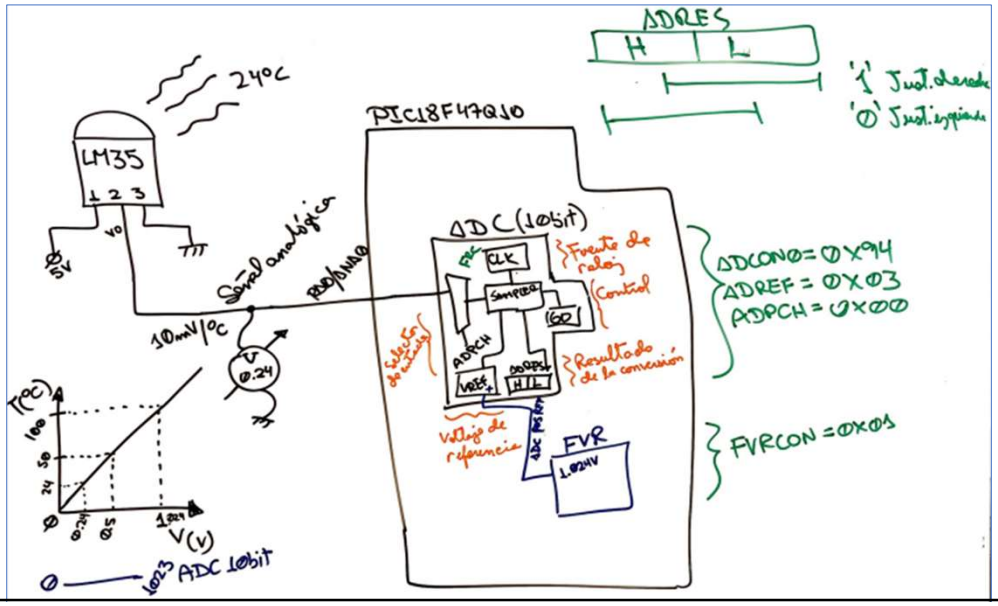
Ejercicio 2026-2: Termómetro LCD y LM35



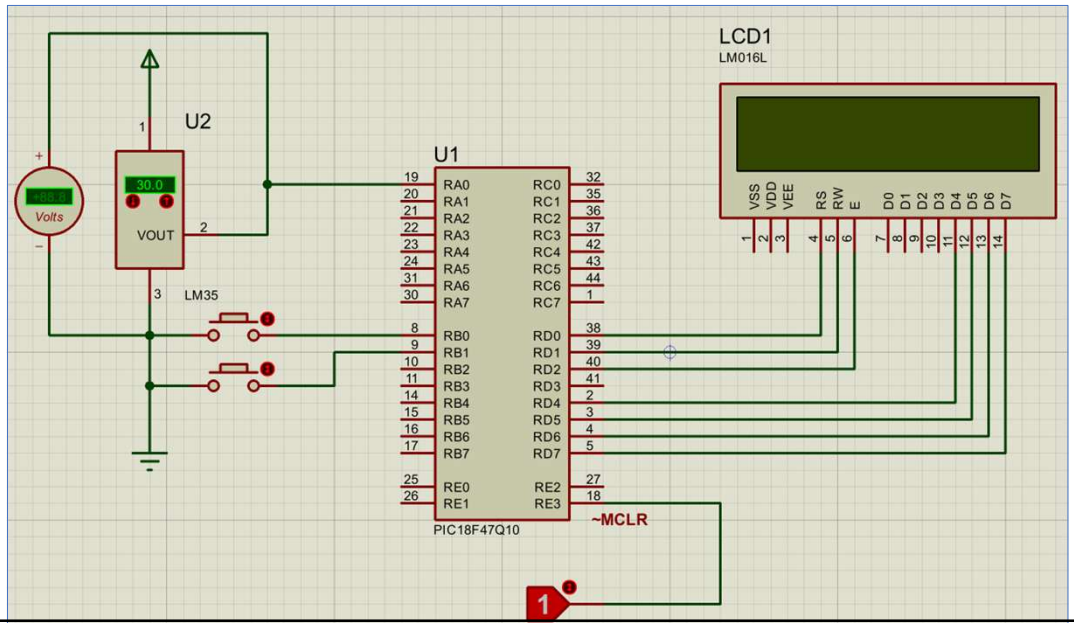
Ejercicio 2026-2: Termómetro LCD y LM35



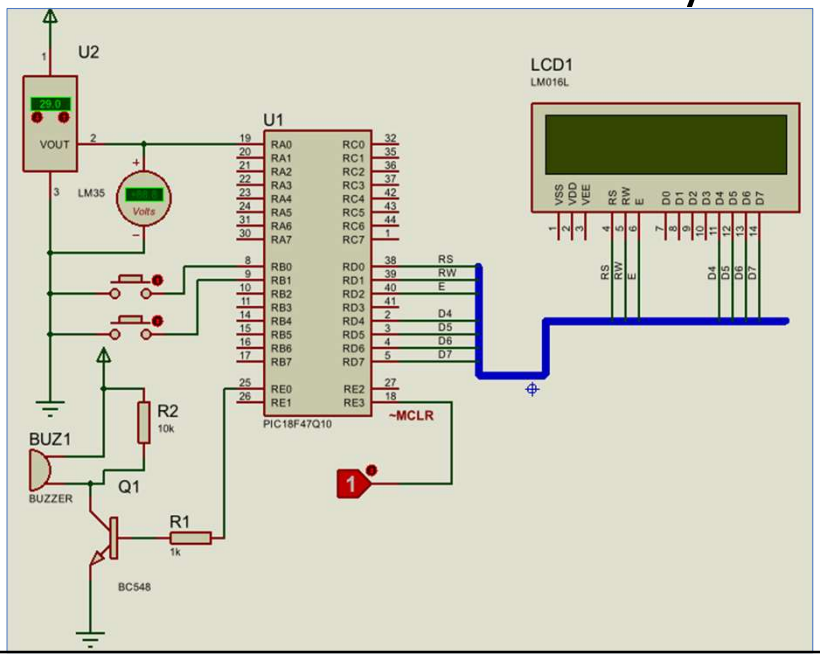
Análisis y configuración del ADC y FVR para leer el sensor LM35



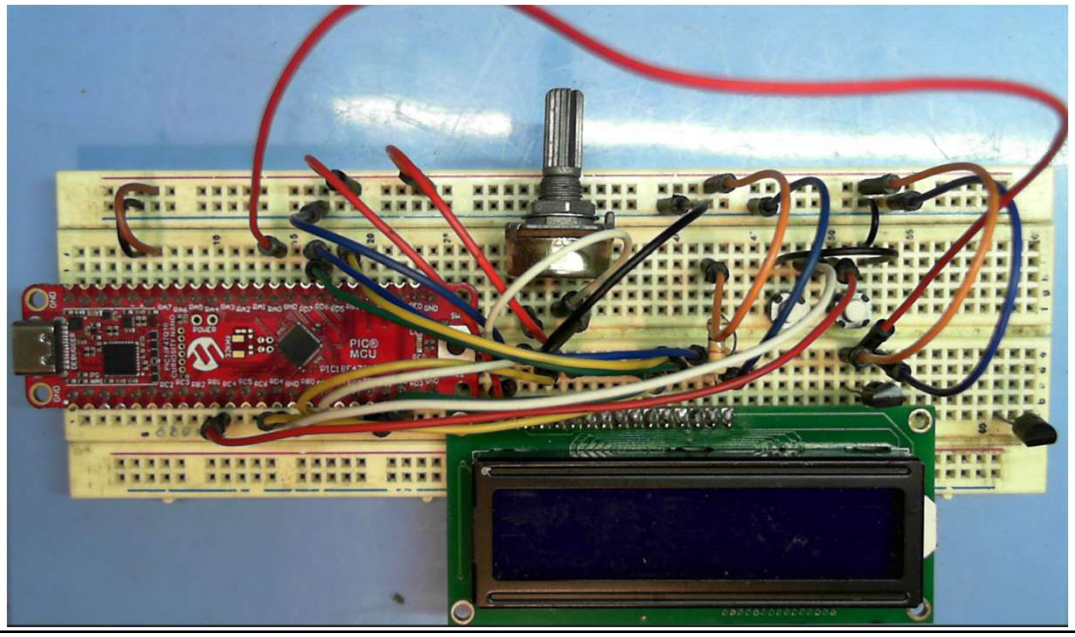
Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35

```
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

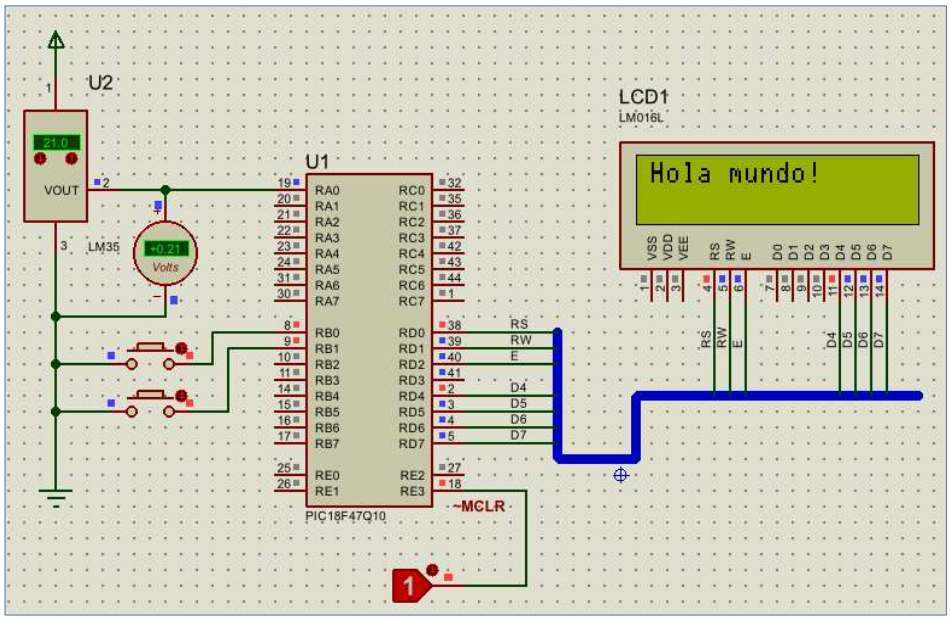
void configuro(void){
    //conf mod de osc
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //conf de las E/S
    ANSELB = 0xFC;
    WPUB = 0x03;
}

void main(void) {
    configuro();
    LCD_INIT();
    while(1){
        POS_CURSOR(1,0);
        ESCRIBE_MENSAJE("Hola mundo!",11);
        __delay_ms(2000);
    }
}
```

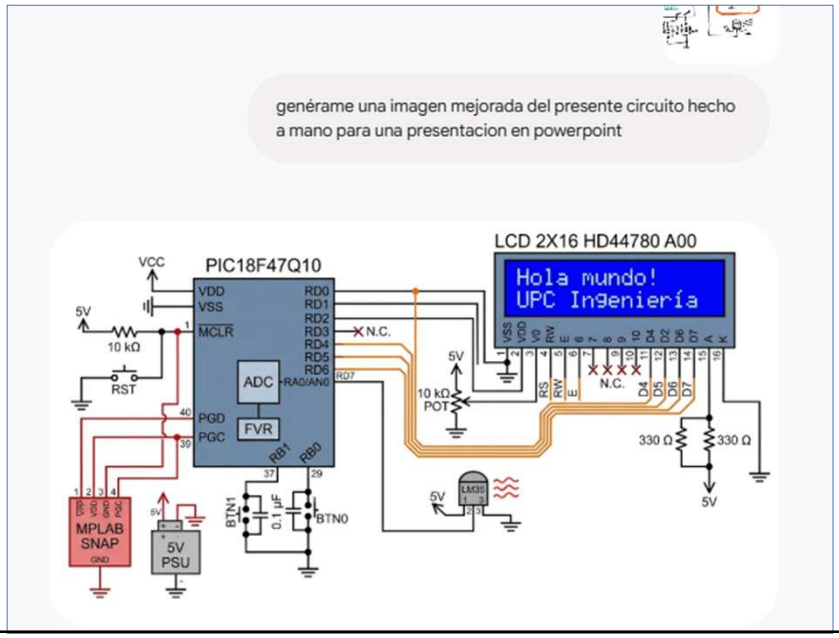
Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35

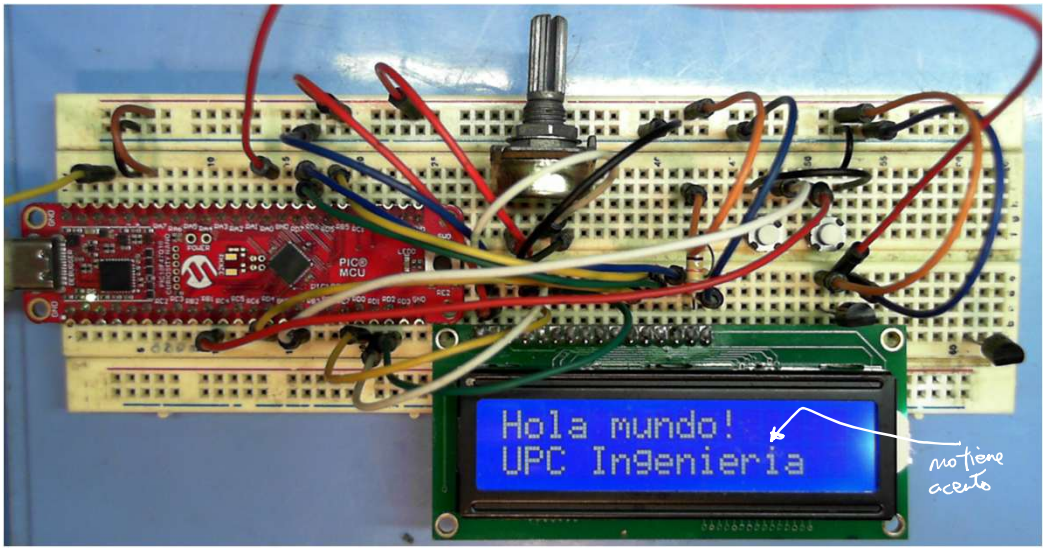
```
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

void configuro(void){
    //conf modulo oscilador
    OSCCON1 = 0x60; //NOSC=HFINTOSC,NDIV=1:1
    OSCFRQ = 0x02;  //HFINTOSC a 4MHz
    OSCEN = 0x40;   //HFINTOSC enabled
    //conf las E/S
    /*RA0 ya es entrada analogica*/
    /*RD lo configura la libreria LCD*/
    ANSELB = 0xFC;  //RB(0:1) digitales
    WPUB = 0x03;    //RB(1:0) weak pullup enabled
    //inicializacion del LCD
    LCD_INIT();
}

void main(void) {
    configuro();
    while(1){
        POS_CURSOR(1,0);
        ESCRIBE_MENSAJE2("Hola mundo!");
        POS_CURSOR(2,0);
        ESCRIBE_MENSAJE2("UPC Ingenieria");
        __delay_ms(1000);
    }
}
```

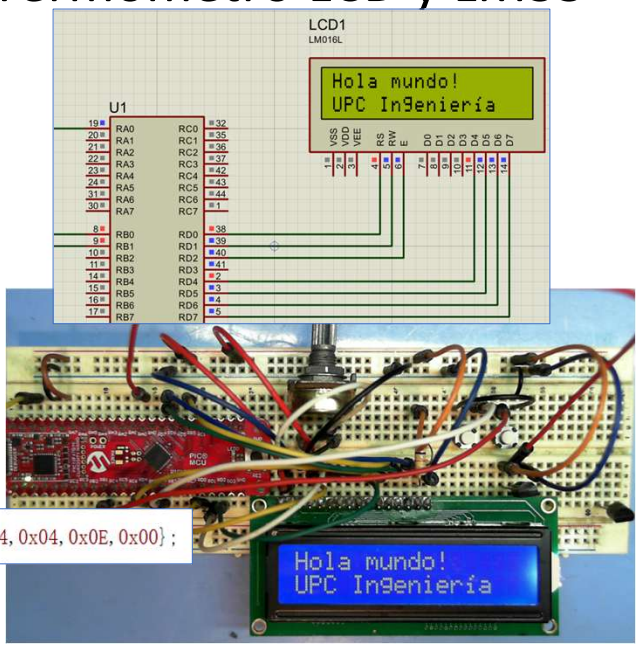
NOTA: Las cadenas de caracteres a enviar al LCD no deben de
contener símbolos de la tabla extendida ASCII, no se visualizarán correctamente debido a que el ROM de caracteres dla tabla ASCII en 7 bits

Ejercicio 2026-2: Termómetro LCD y LM35



Ejercicio 2026-2: Termómetro LCD y LM35

D4	D3	D2	D1	D0	
			■		0x02
		■			0x04
					0x00
	■	■			0x0C
		■			0x04
		■			0x04
	■	■	■		0x0E
					0x00

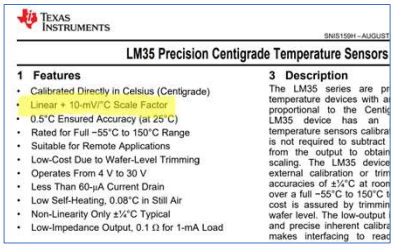



```
unsigned char i_acento[]={0x02, 0x04, 0x00, 0x0C, 0x04, 0x04, 0x0E, 0x00};
```

```
GENERACARACTER(i_acento, 0);
```

```
ENVIA_CHAR(0);
```

Conexión del LM35 con el ADC del PIC18F47Q10



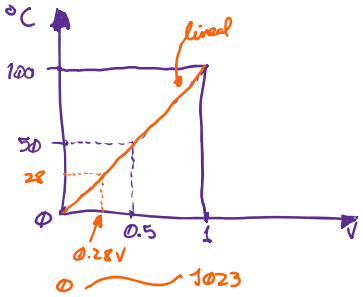
LM35 Precision Centigrade Temperature Sensors

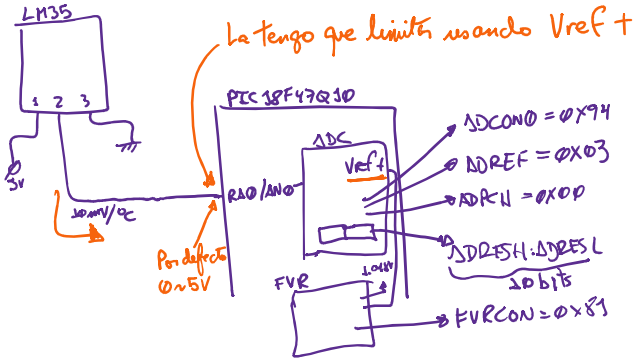
1 Features

- Calibrated Directly in Celsius (Centigrade)
- Linear + 10-mV/°C Scale Factor
- 0.5°C Ensured Accuracy (at 25°C)
- Rated for Full −55°C to 150°C Range
- Suitable for Remote Applications
- Low-Cost Due to Wafer-Level Trimming
- Operates From 4 V to 30 V
- Less Than 60-µA Current Drain
- Low Self-Heating, 0.08°C in Still Air
- Non-Linearity Only ±¼°C Typical
- Low-Impedance Output, 0.1 Ω for 1-mA Load

3 Description

The LM35 series are pr temperature devices with a proportional to the Centi LM35 device has an temperature sensors calibra is not required to subtract from the output to obtain scaling. The LM35 device external calibration or trim accuracies of ±¼°C at room over a full −55°C to 150°C t cost is assured by trimmin wafer level. The low-output and precise inherent calibra makes interfacing to read





Conexión del LM35 con el ADC del PIC18F47Q10

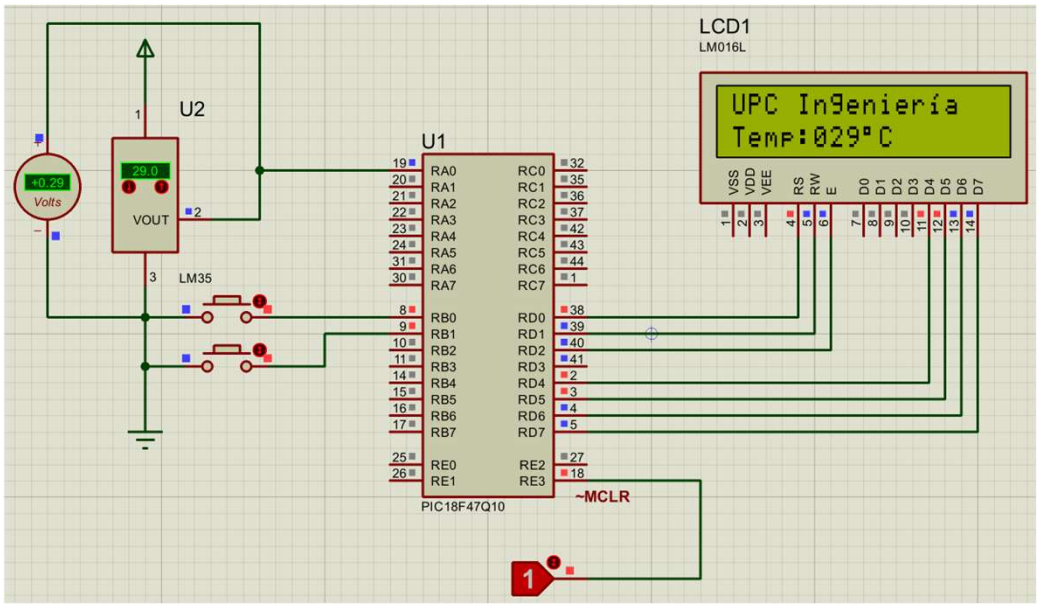
```c
#include <xc.h>
#include "cabecera.h"
#include "LCD.h"
#define _XTAL_FREQ 4000000UL

unsigned char i_acento[]={0x02, 0x04, 0x00, 0x0C, 0x04, 0x04, 0x0E, 0x00};

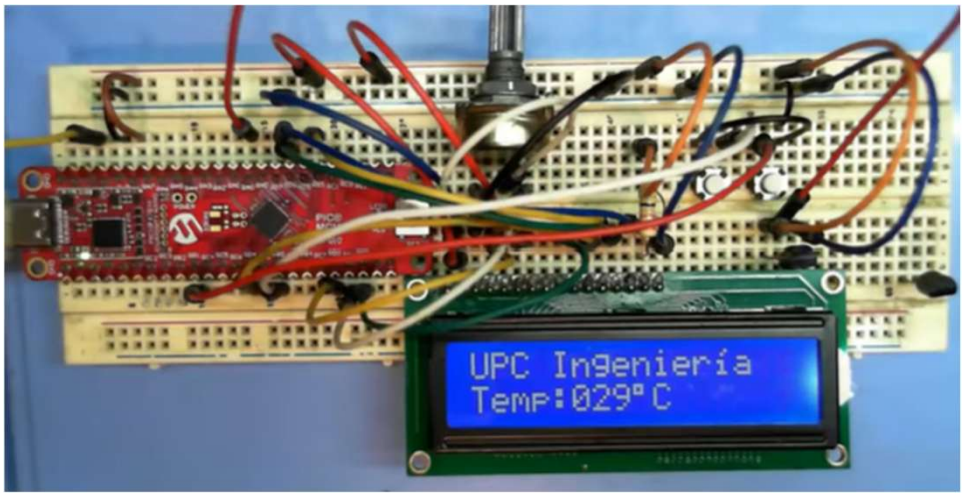
void configuro(void){
    //conf mod de osc
    OSCCON1 = 0x60;
    OSCFRQ = 0x02;
    OSCEN = 0x40;
    //conf de las E/S
    ANSELB = 0xFC;
    WPUB = 0x03;
    //conf del ADC para que funcione el LM35
    ADCON0 = 0x841;
    ADREF = 0x03;
    ADPCH = 0x00;
    FVRCON =0x81;
}
```

```c
void main(void) {
    configuro();
    LCD_INIT();
    GENERACARACTER(i_acento,0);
    while(1){
        POS_CURSOR(1,0);
        ESCRIBE_MENSAJE2("UPC Ingenier");
        ENVIA_CHAR(0);
        ENVIA_CHAR('a');
        ADCON0bits.GO_nDONE = 1;
        while(ADCON0bits.GO_nDONE == 1);
        unsigned int resultado = (ADRESH << 8) + ADRESL;
        POS_CURSOR(2,0);
        ESCRIBE_MENSAJE2("Temp:");
        LCD_ESCRIBE_VAR_INT(resultado/10, 3);
        ENVIA_CHAR(0xDF);
        ENVIA_CHAR('C');
        __delay_ms(1000);
    }
}
```

Conexión del LM35 con el ADC del PIC18F47Q10



Conexión del LM35 con el ADC del PIC18F47Q10



Fin de la sesión